



COMPILER

Introduction

- Programming languages are notations for describing computations to people and to machines.
- Before a program can be run, it first must be translated into a form in which it can be executed by a computer. The software systems that do this translation are called compilers.
- A compiler is a program that can read a program in one language - the source language and translate it into an equivalent program in another language - the target language.

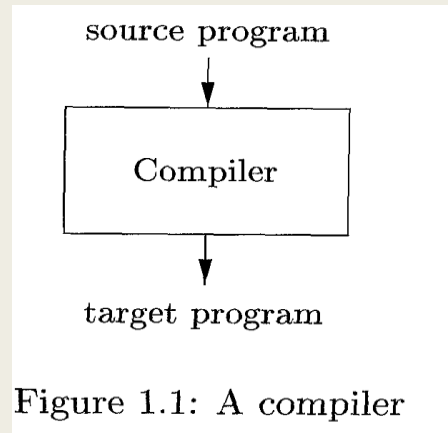


Figure 1.1: A compiler

- If the target program is an executable machine-language program, it can then be called by the user to process inputs and produce outputs

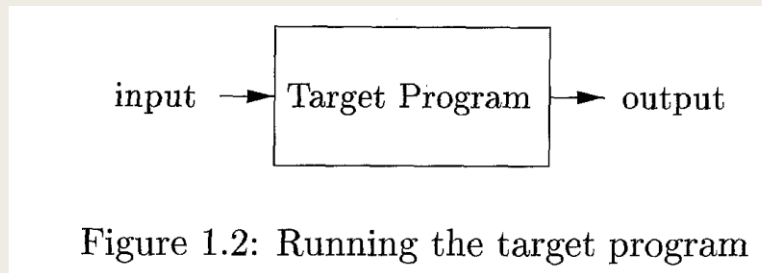


Figure 1.2: Running the target program

- An interpreter is another common kind of language processor. Instead of producing a target program as a translation, an interpreter appears to directly execute the operations specified in the source program on inputs supplied by the user.

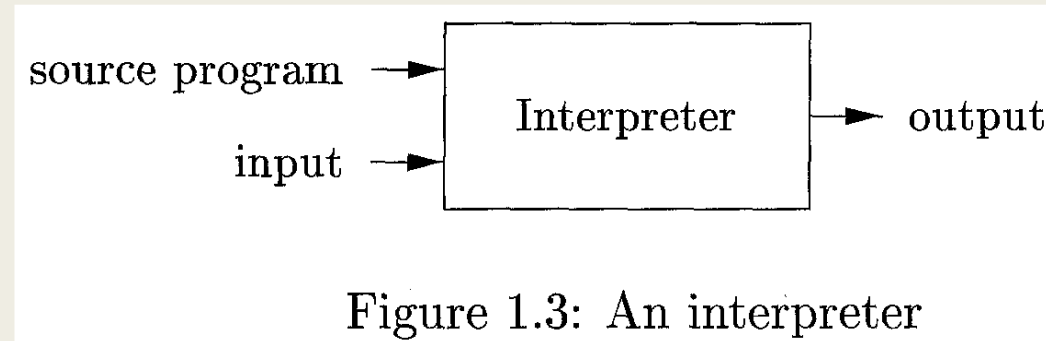


Figure 1.3: An interpreter

- The machine-language target program produced by a compiler is usually much faster than an interpreter at mapping inputs to outputs .

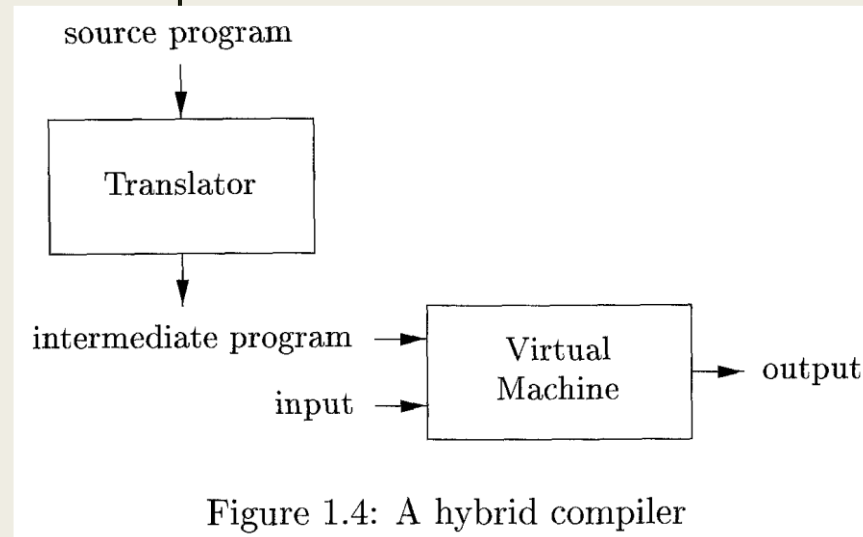


Figure 1.4: A hybrid compiler



A Language Processing System

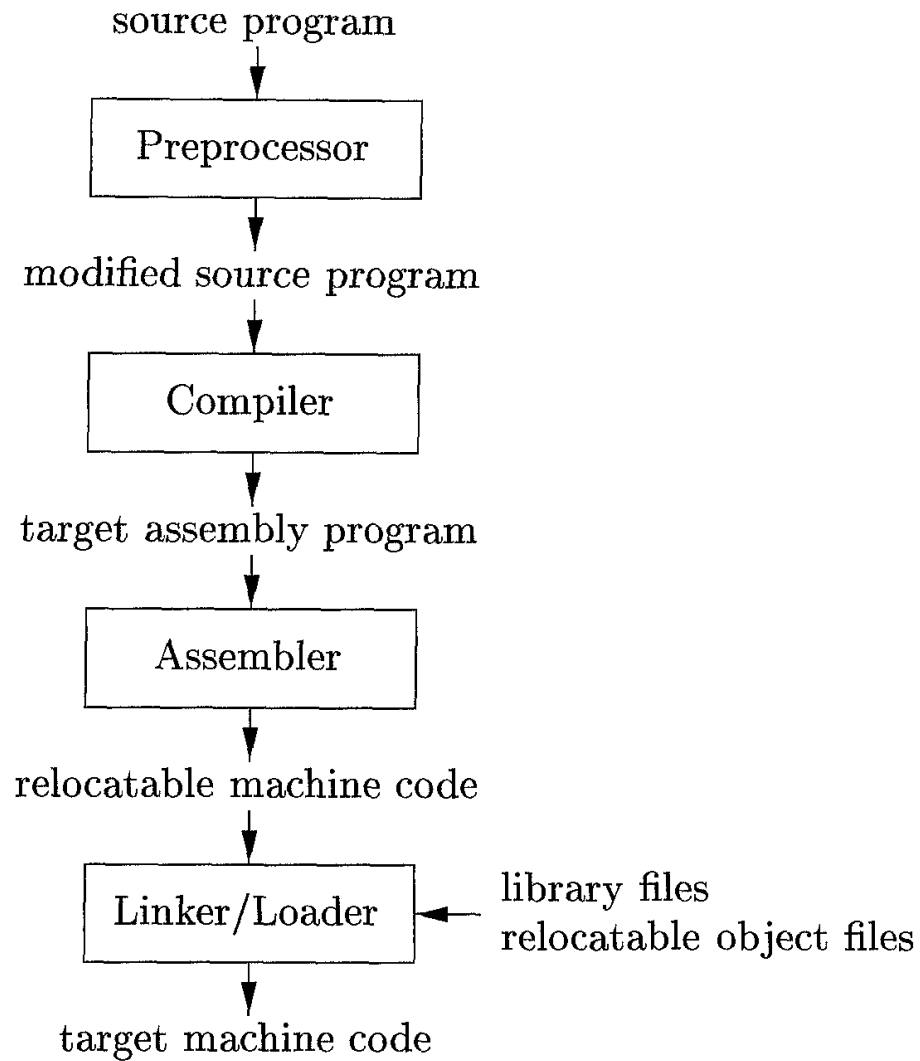
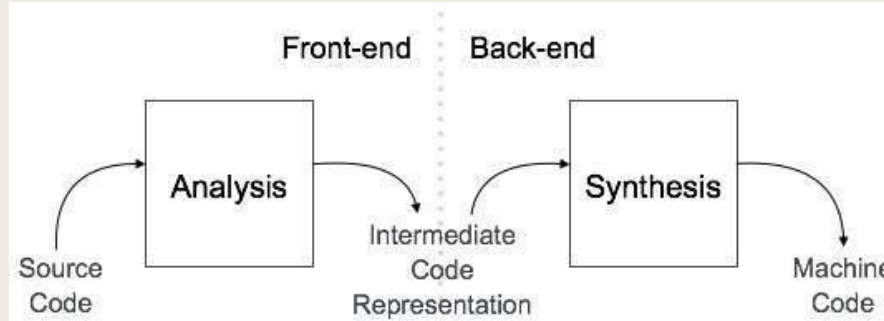


Figure 1.5: A language-processing system

Analysis-Synthesis Model of Compiler



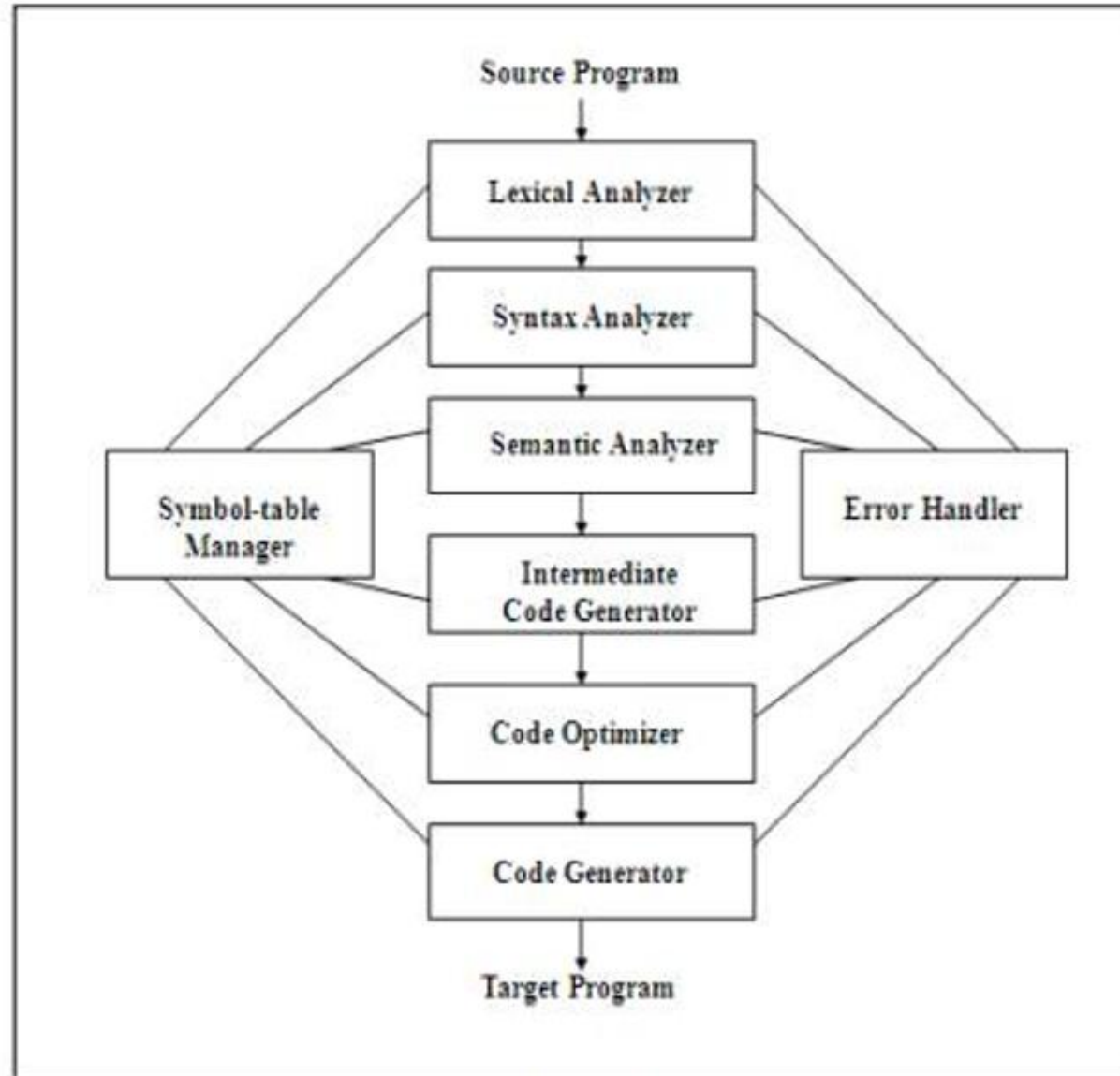
■ Analysis

- The analysis part breaks up the source program into constituent pieces and imposes a grammatical structure on them.
- It then uses this structure to create an intermediate representation of the source program.
- If the analysis part detects that the source program is either syntactically ill formed or semantically unsound, then it must provide informative messages, so the user can take corrective action.
- The analysis part also collects information about the source program and stores it in a data structure called a symbol table, which is passed along with the intermediate representation to the synthesis part.

■ Synthesis

- The synthesis part constructs the desired target program from the intermediate representation and the information in the symbol table.
- The analysis part is often called the front end of the compiler; the synthesis part is the back end.

Phases of Compiler



Phases of Compiler

■ Lexical Analysis

- *The first phase of a compiler is called lexical analysis or scanning.*
- *The lexical analyzer reads the stream of characters making up the source program and groups the characters into meaningful sequences called lexemes.*
- *For each lexeme, the lexical analyzer produces as output a token of the form:*
<token-name, attribute-value>
- *In the token, the first component token-name is an abstract symbol that is used during syntax analysis, and the second component attribute-value points to an entry in the symbol table for this token.*

■ Syntax Analysis

- *The second phase of the compiler is syntax analysis or parsing.*
- *The parser uses the first components of the tokens produced by the lexical analyzer to create a tree-like intermediate representation that depicts the grammatical structure of the token stream.*
- *A typical representation is a syntax tree in which each interior node represents an operation and the children of the node represent the arguments of the operation.*

■ Semantic Analysis

- *The semantic analyzer uses the syntax tree and the information in the symbol table to check the source program for semantic consistency with the language definition.*
- *It also gathers type information and saves it in either the syntax tree or the symbol table, for subsequent use during intermediate-code generation.*
- *An important part of semantic analysis is type checking, where the compiler checks that each operator has matching operands.*

■ Intermediate Code Generation

- *This phase generates an intermediate representation of the source code that can be easily translated into machine code.*
- *This intermediate representation should have two important properties: it should be easy to produce and it should be easy to translate into the target machine.*
- *One form of intermediate representation is called three-address code, which consists of a sequence of assembly-like instructions with three operands per instruction. Each operand can act like a register.*
- *Each three-address assignment instruction has at most one operator on the right side.*
- *The compiler must generate a temporary name to hold the value computed by a three-address instruction.*
- *Some "three-address instructions" have fewer than three operands.*

■ Code Optimization

- *The code optimizer attempts to improve the quality of Intermediate code, so that faster running target code will result.*
- *The Code optimizer considers the factors like algorithm, memory, running time of the target program by applying code improving transformations.*
- *The Code optimization is optional and can be performed:*
 - *Before Code generation – transformation applied over the intermediate representation of the source program, called Pre-Optimization or Machine Independent Optimization.*
 - *After Code Generation – transformation applied on the target code, called post optimization or machine dependent optimization.*

■ Code Generation

- *The code generator takes as input an optimized Intermediate representation of the source program and maps it into the target language.*
- *If the target language is machine code, registers or memory locations are selected for each of the variables used by the program. Then, the intermediate instructions are translated into sequences of machine instructions that perform the same task.*

■ Symbol Table Management

- *The Symbol Table is a data structure containing a record for each identifier with fields for attributes for the same.*
- *It is an essential function of compiler to record the symbols used in the source program and collects the information about various attributes (like storage, type, scope) of each symbol.*

■ Error Handler

- *One of the most important functions of a compiler is the detection and reporting of errors in the source program.*
- *The error message should allow the programmer to determine exactly where the errors have occurred. Errors may occur in all the phases of a compiler.*
- *Whenever a phase of the compiler discovers an error, it must report the error to the error handler, which issues an appropriate diagnostic message.*

Example

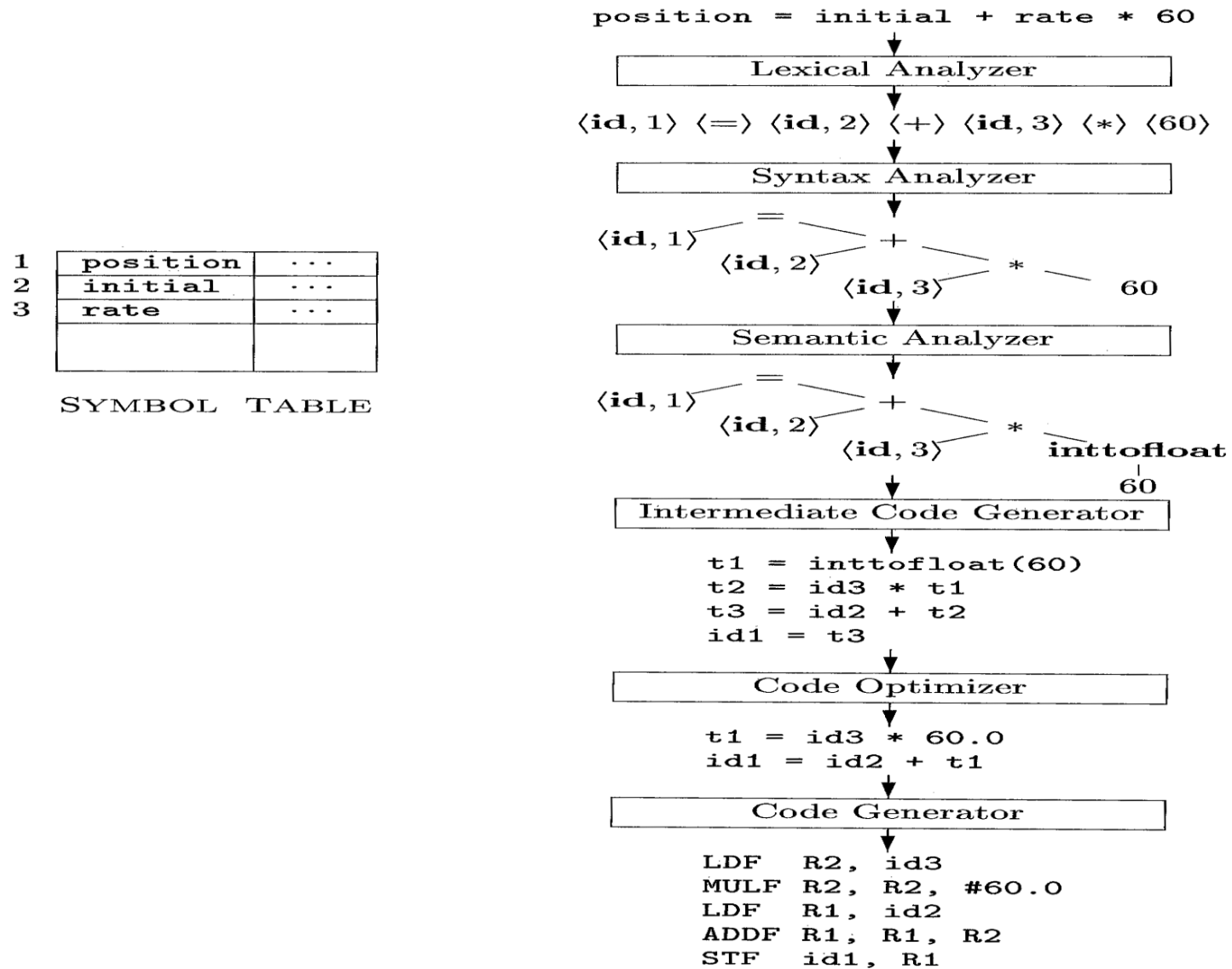


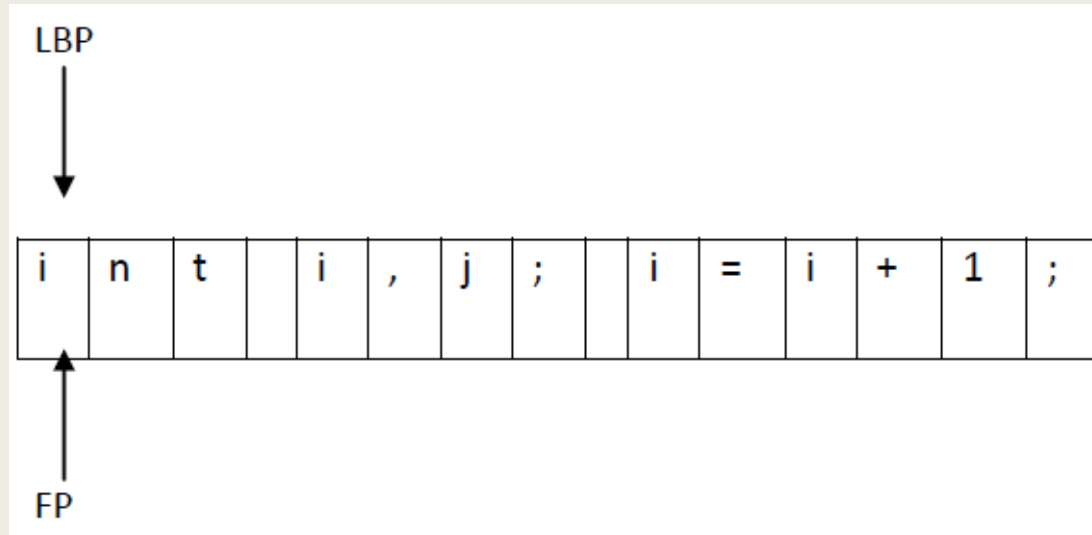
Figure 1.7: Translation of an assignment statement

Compiler Construction/Writing Tools

- Parser generators automatically produce syntax analyzers from a grammatical description of a programming language.
- Scanner generators produce lexical analyzers from a regular-expression description of the tokens of a language.
- Syntax-directed translation engines produce collections of routines for walking a parse tree and generating intermediate code.
- Code-generator generators produce a code generator from a collection of rules for translating each operation of the intermediate language into the machine language for a target machine.
- Data-flow analysis engines facilitate the gathering of information about how values are transmitted from one part of a program to each other part.
- Compiler-construction toolkits provide an integrated set of routines for constructing various phases of a compiler.

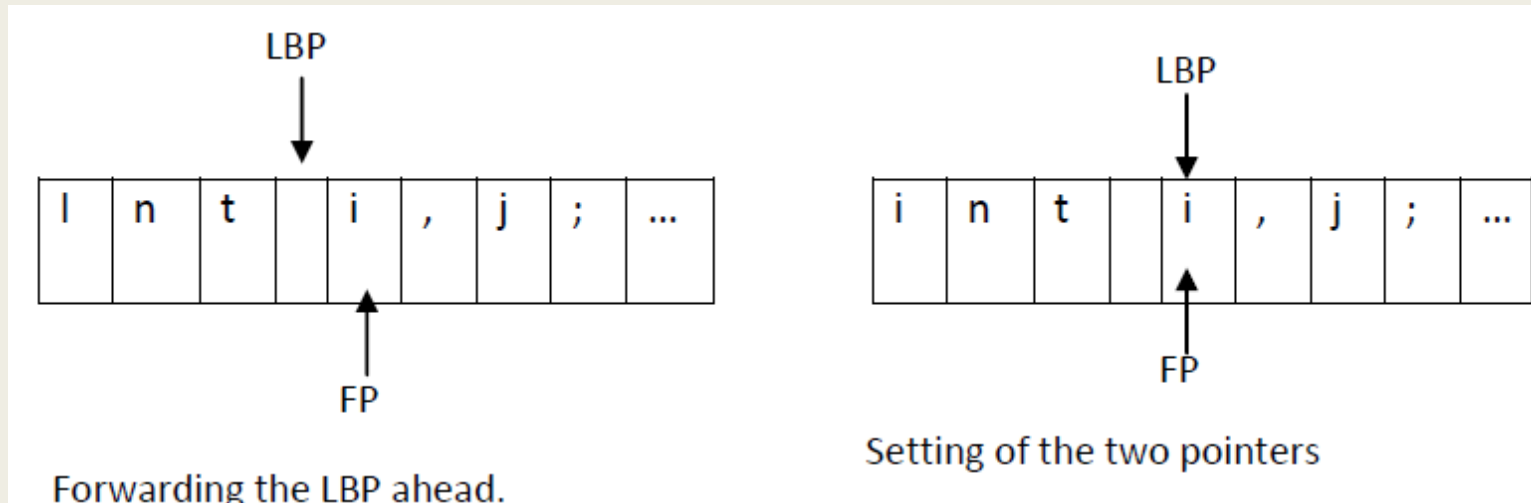
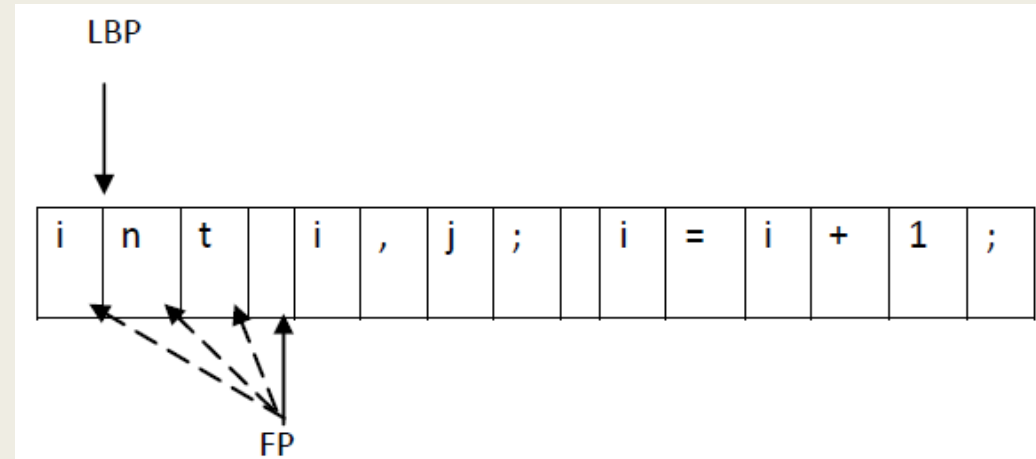
Input Buffering

- Using a pair of buffers (instead of single buffer) cyclically and ending each buffer's contents with a sentinel character that warns of its ends are the two techniques that accelerate the process of buffering.
- The lexical analyzer scans the input string from left to right one character at a time. It uses two pointers: Lexeme_begin_pointer (LBP) and Forward_ptr (FP) to keep track of the portion of the input scanned. Initially both the pointers point to the first character of the input string.



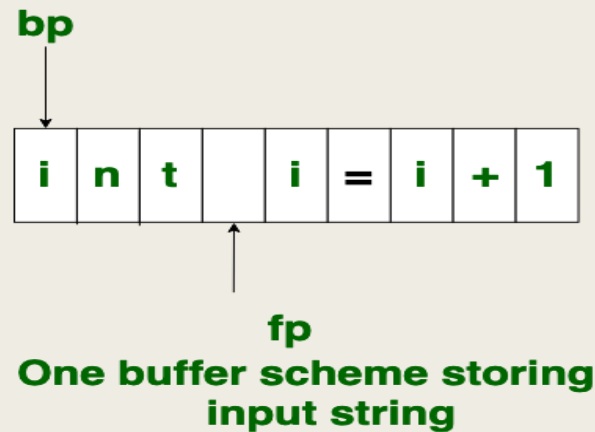
- The forward_ptr moves ahead to search for end of lexeme. As soon as the white space is encountered it indicates end of lexeme. In the above example as soon as the forward_ptr (FP) encounters a blank space the lexeme "int" is identified.

- At the moment the forward_ptr is located in the blank space. So the forward_ptr will ignore the blank space and it will move ahead. Then both the Lexeme_begin_ptr (LBP) and the forward_ptr (FP) will be set for the next lexeme.



■ One Buffer Scheme

- One buffer is used to store the input.
- Problem with this scheme is that if lexeme is very long then it crosses the buffer end; to scan rest of the lexeme the buffer has to be reloaded, that results in overwriting the first part of lexeme



■ Two Buffer Scheme

- To overcome the problem of one buffer scheme, two buffers are used to store the input.

